RAG Research Assistant

Complete Interview Preparation Guide

College Knowledge Assistant • Retrieval-Augmented Generation
Python · Sentence Transformers · ChromaDB · OpenRouter · Streamlit
Prepared for Kartik Bansal

> **How to use this document.** Part 1 and Part 2 teach you the project so you can explain it to anyone. Part 3 and 4 give you the vocabulary — every tool and every term, in plain words. Part 5 is roughly 110 interview questions with answers. Part 6 is the honest list of weaknesses in your code, so you are never caught off guard. Part 7 is what to revise the night before.

Contents

Part 1 — The Project in Plain English

1.1 What does it actually do?

You built a chatbot that answers questions about a college handbook, and it only answers from what is actually written in that handbook.

You give it a PDF — the college handbook covering admissions, fees, hostel rules, library rules, scholarships, placements. Then anyone can ask a normal English question like "*What is the hostel curfew time?*" and it replies with the real answer from the handbook, not a made-up one.

1.2 The problem it solves

A plain LLM like ChatGPT has never read *your* college handbook. If you ask it about your hostel curfew, it does one of two things — it says "I don't know," or worse, it **invents** a confident-sounding answer. That invention is called a **hallucination**, and it is the single biggest problem with using LLMs on private documents.

You cannot fix this by pasting the whole handbook into the prompt every time. Documents are too big — an LLM has a limited **context window** (how much text it can read at once), and sending 200 pages on every question would be slow and expensive.

RAG solves both problems. Instead of sending the whole document, you send only the 3 most relevant paragraphs. The LLM reads those and answers. Small, fast, cheap, and grounded in real text.

> **The open-book exam analogy — use this in the interview, it lands well.**
>
> A **plain LLM** is a student writing an exam from memory. If they never studied your syllabus, they guess.
>
> **RAG** is the same student writing an *open-book* exam. Before answering, they flip to the right page, read it, and then answer using what's on that page.
>
> Your project builds the index at the back of the book (indexing) and does the page-flipping (retrieval). The LLM just reads and writes the final answer.

1.3 The two phases

Every RAG system has exactly two phases. If you remember nothing else, remember this split — interviewers ask it constantly.

Phase	When it runs	What happens
Indexing (Ingestion)	Once, offline, before anyone asks anything	Read the PDF → cut it into chunks → convert each chunk into numbers (embeddings) → store them in a vector database. This is `index.py`.
Retrieval + Generation (Query time)	Every single time a user asks a question	Convert the question into numbers → find the closest chunks → paste them into a prompt → send to the LLM → return the answer. This is `app.py` / `streamlit_app.py`.

One-line version to say out loud: "Indexing happens once and is slow; retrieval happens on every question and must be fast. That's why I precompute the embeddings and store them, instead of embedding the document each time."

1.4 Your 90-second pitch

Interviews almost always open with "*tell me about your project.*" Have this ready word for word. Do not ramble past 90 seconds — leave them room to ask.

"I built a RAG-based question-answering assistant over a college handbook. The problem I was solving is that a normal LLM has never seen a private document, so it either refuses to answer or it hallucinates."

"The system has two phases. In the indexing phase, I extract text from the PDF with PyPDF, split it into 500-character overlapping chunks, convert each chunk into a 384-dimensional embedding using the all-MiniLM-L6-v2 sentence transformer, and store those vectors in ChromaDB."

"At query time, I embed the user's question with the exact same model, run a top-K similarity search in ChromaDB to pull the 3 most relevant chunks, inject them into a prompt as context, and send that to an LLM through OpenRouter. The prompt explicitly instructs the model to answer only from the provided context and to say 'I could not find that' if the answer isn't there — that's my main guard against hallucination."

"I deliberately built it without LangChain so I understood every step myself. The front end is Streamlit with a chat interface and a sources expander so you can see which passages the answer came from, and it's deployed on Streamlit Community Cloud."

Two phrases in that pitch that earn follow-up questions — make sure you want them: "without LangChain" (they'll ask why, and whether you know LangChain) and "guard against hallucination" (they'll ask what else you'd do). Both are covered later in this document.

Part 2 — How It Works, Step by Step

Let's follow one real question — "*What is the hostel curfew time?*" — all the way from PDF to answer. Eight stages. For each one: what it does, why it's needed, your actual code, and what breaks if you get it wrong.

Stage 1 — PDF to raw text `pdf_processor.py`

What it does

Opens the PDF and pulls out all the text as one long Python string.

Why it's needed

A PDF is a layout format — it stores "put this letter at this x-y position on the page." Computers can't search meaning in that. You need plain text first.

```
from pypdf import PdfReader

def extract_text(pdf_path):
    reader = PdfReader(pdf_path)
    text = ""
    for page in reader.pages:
        text += page.extract_text() + "\n"
    return text
```

Simple example

A 12-page handbook becomes one string about 30,000 characters long, starting `"COLLEGE HANDBOOK 2024\nAdmissions\nB.Tech eligibility requires..."`

What breaks

- **Scanned PDFs return nothing.** If the PDF is a photo of a page, there is no text layer — `extract_text()` returns an empty string. You would need **OCR** (Optical Character Recognition, e.g. Tesseract) for that. *This is a very common interview question.*
- **Tables get mangled.** Columns flatten into one line, so a fee table can come out scrambled.
- **Multi-column layouts** can interleave text from both columns.

Stage 2 — Text to chunks `chunker.py`

What it does

Cuts the one long string into small overlapping pieces of 500 characters each, where each piece repeats the last 100 characters of the previous piece.

Why chunking is needed — three reasons

1. **The embedding model has a size limit.** all-MiniLM-L6-v2 only reads 256 tokens (~200 words) at a time. Anything longer is silently cut off.
2. **Precision.** If you embedded the whole 30,000-character document as one vector, it would be an average of every topic — admissions, fees, hostel, library all blurred together. It would match everything vaguely and nothing well.

3. **Context window and cost.** You only send the LLM 3 small chunks, not the whole book.

Why overlap is needed

This is the question interviewers love, because the answer is so easy to picture.

> Imagine the handbook says: "*...students must return to the hostel by | 10:00 PM on weekdays.*"
>
> If a chunk boundary falls at the **|**, then chunk 4 ends with "students must return to the hostel by" and chunk 5 starts with "10:00 PM on weekdays." **Neither chunk contains the full answer.** Chunk 4 has the question but no time; chunk 5 has a time but no idea what it refers to.
>
> With 100 characters of overlap, chunk 5 *also* begins with "...must return to the hostel by 10:00 PM on weekdays" — so the complete fact survives in at least one chunk. **Overlap is insurance against cutting a fact in half.**

```
def chunk_text(text, chunk_size=500, overlap=100):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start += chunk_size - overlap   # advance 400, not 500
    return chunks
```

Trace the maths — be ready to do this on a whiteboard

- Chunk 0 = characters 0–500. Then `start = 0 + 400 = 400`
- Chunk 1 = characters 400–900. Then `start = 800`
- Chunk 2 = characters 800–1300 ... and so on
- **Stride** = `chunk_size - overlap` = 400. Characters 400–500 appear in both chunk 0 and chunk 1.

> **Important honesty point.** Your README calls this "recursive text chunking." It is **not** recursive — it is **fixed-size character chunking**. Recursive chunking (what LangChain's `RecursiveCharacterTextSplitter` does) tries to split on paragraph breaks first, then sentences, then words, and only cuts mid-word as a last resort. Yours cuts blindly at exactly 500 characters, even mid-word. Call it "fixed-size chunking with overlap" in the interview. If they ask about recursive chunking, say you know the difference and it's your top improvement — that turns a gap into a strength.

Stage 3 — Chunks to embeddings `embedding_model.py`

What an embedding is

An embedding turns a piece of text into a list of numbers that captures its **meaning**. Your model produces **384 numbers** for every chunk.

> **The map analogy — the clearest way to explain embeddings.**
>
> Think of a giant map where every sentence gets a coordinate. Sentences about *hostel rules* land in one neighbourhood. Sentences about *library fines* land in a different neighbourhood, far away.
>
> A normal map needs 2 numbers (latitude, longitude). Meaning is far more complicated than a flat map, so this one needs **384 numbers** per point. That's all "384-dimensional" means — 384 coordinates instead of 2.
>
> **The payoff:** "hostel curfew" and "what time must students return to their rooms" share almost no words, but they land at nearly the same coordinate — because they mean the same thing.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

def generate_embeddings(chunks):
    embeddings = model.encode(chunks)
    return embeddings   # shape: (number_of_chunks, 384)
```

What happens inside `model.encode()` — know these three steps

1. **Tokenization.** Text is split into tokens (word-pieces). "curfew" might become `["cur", "##few"]`. Each token becomes an ID number.
2. **Transformer layers.** 6 layers of a BERT-style network process the tokens. Each token comes out with its own 384-number vector, adjusted by the words around it — so "bank" in "river bank" and "bank account" get different vectors. This is what **attention** buys you.
3. **Mean pooling + normalization.** You have one vector per token but need one per sentence, so you average them all — that's **mean pooling**. Then the result is scaled to length 1 — that's **L2 normalization**.

> **A detail that will impress.** all-MiniLM-L6-v2 has a `Normalize` layer built into its pipeline, so every vector it outputs already has length exactly 1. This matters: for unit-length vectors, ranking by Euclidean (L2) distance and ranking by cosine similarity give **identical results**, because `||a - b||² = 2 - 2·cos(a, b)`. Very few candidates know this. See Stage 6.

Why "all-MiniLM-L6-v2" specifically

Property	Value	Why it matters
Layers	6	"L6" in the name. BERT-base has 12 — half the layers means roughly twice the speed.
Output dimensions	384	BERT-base gives 768. Half the size = half the storage and faster search.
Parameters	~22 million	Small enough to run on CPU with no GPU. Critical for free-tier deployment.
Max input	256 tokens	~200 words. Your 500-character chunks (~85 words) fit comfortably.
Model size	~90 MB	Downloads in seconds on Streamlit Cloud's first boot.

Say this: "I chose it because it's the best speed-to-quality trade-off for CPU-only deployment. It's small, fast, free, runs locally with no API cost, and for a single handbook the quality is more than enough. If I needed higher accuracy I'd move to `all-mpnet-base-v2` (768-dim, slower but better) or a paid API embedding model."

Stage 4 — Store in ChromaDB vector_store.py

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(name="research_docs")

def store_chunks(chunks, embeddings):
    ids = [f"chunk_{i}" for i in range(len(chunks))]
    collection.add(
        ids=ids,
        documents=chunks,
        embeddings=embeddings.tolist()
    )
```

Each record stores three things together: an **ID** (`chunk_0`), the **original text** (so you can show it to the LLM later), and the **384-number vector** (so you can search by meaning).

`PersistentClient` writes to disk, so the index survives a restart. The alternative, `EphemeralClient`, keeps everything in memory and loses it when the program exits. `.tolist()` is needed because the model returns a NumPy array and Chroma expects plain Python lists.

Stage 5 — Question to embedding retriever.py

```
query_embedding = model.encode(query)
```

> **The single most important rule in this entire project:** the question **must** be embedded with the **exact same model** as the chunks. Different models produce coordinates on completely different maps — comparing them gives meaningless results. If you ever swap the embedding model, you must **re-index every document**. Interviewers ask this to check you actually understand embeddings.

Stage 6 — Similarity search retriever.py

```
results = collection.query(
    query_embeddings=[query_embedding.tolist()],
    n_results=k          # k = 3
)
```

Chroma compares the question's vector against all stored chunk vectors and returns the **k closest**. Going back to the map analogy: you drop a pin where the question lands, and ask "which 3 chunks are nearest to this pin?"

How closeness is measured

Metric	Plain-English meaning
Cosine similarity	The *angle* between two arrows. Ignores length, cares only about direction. Range −1 to 1; 1 means identical meaning.
Euclidean (L2) distance	Straight-line distance between two points. Smaller = more similar. **This is ChromaDB's default**, and your code doesn't change it.
Dot product	Combines direction and magnitude. Fast, used when vectors are already normalized.

> **How to answer "what distance metric do you use?" — the honest, impressive version.**
> "ChromaDB defaults to squared L2, and I didn't override it, so technically it's L2. But my embedding model outputs unit-normalized vectors, and for unit vectors L2 and cosine produce the identical ranking — so the retrieved results are the same either way. If I wanted cosine explicitly I'd pass `metadata={"hnsw:space": "cosine"}` when creating the collection."

Why it's fast — HNSW

Comparing the query against every single vector is called a **brute-force** or **exact** search. With 23 chunks that's instant, but with 10 million chunks it's far too slow. So vector databases use **ANN — Approximate Nearest Neighbour** search. Chroma's algorithm is **HNSW** (Hierarchical Navigable Small World).

> **HNSW in one picture.** It's like a country's road network. To get from Delhi to a specific house in Mumbai you don't check every house in India. You take the highway to Mumbai (top layer, big jumps), then the main road to the neighbourhood (middle layer), then the small streets to the exact house (bottom layer). HNSW builds those layers over your vectors.
>
> **The trade-off:** "approximate" means it can occasionally miss the true nearest neighbour. You give up a tiny bit of accuracy for an enormous speed gain. That trade-off is the whole point of ANN.

Stage 7 — Build the augmented prompt `llm.py`

The 3 retrieved chunks are joined together and pasted into a prompt template alongside the question. This step is what the "A" in RAG — **Augmented** — refers to.

```
context = "\n".join(results["documents"][0])

prompt = f"""
You are a helpful research assistant.
Use ONLY the provided context.
If the answer is not present in the context, say:
'I could not find that information in the provided documents.'

Context:
{context}

Question:
{question}

Answer:
"""
```

Three deliberate prompt-engineering decisions — be able to defend each

1. **Role assignment** — "You are a helpful research assistant" sets the tone and style.
2. **Grounding constraint** — "Use ONLY the provided context" is the instruction that stops the model from answering from its own training memory.
3. **Explicit fallback** — giving it a sentence to say when the answer is missing. Without this, an LLM under pressure to be helpful will invent something. **Giving the model a licensed way to fail is the single most effective anti-hallucination technique.**

Stage 8 — Generate the answer `llm.py`

```
client = OpenAI(
    api_key=os.getenv("OPENROUTER_API_KEY"),
    base_url="https://openrouter.ai/api/v1"      # the key trick
)

response = client.chat.completions.create(
    model="qwen/qwen3-235b-a22b",
    messages=[{"role": "user", "content": prompt}]
)
return response.choices[0].message.content
```

> **Why the OpenAI library talks to OpenRouter.** OpenRouter deliberately copies OpenAI's API format. So you use the official OpenAI Python SDK but point `base_url` at OpenRouter's server instead. One line changed, and now you can call Llama, DeepSeek, Qwen, Claude or Gemini through the same code. **This is called an OpenAI-compatible API** — worth knowing the term.

The complete trace

```
Question: "What is the hostel curfew time?"
   │
   ├─ 1. model.encode(question)  →  [0.031, -0.184, 0.442, ... ]   (384 numbers)
   │
   ├─ 2. collection.query(..., n_results=3)
   │        Chroma walks the HNSW graph, compares vectors
   │        ↓
   │     chunk_11  "...hostel gates close at 10:00 PM on weekdays..."
   │     chunk_12  "...late entry requires written permission from..."
   │     chunk_07  "...hostel fee is Rs 45,000 per year including..."
   │
   ├─ 3. context = chunk_11 + "\n" + chunk_12 + "\n" + chunk_07
   │
   ├─ 4. prompt = instructions + context + question
   │
   ├─ 5. OpenRouter → Qwen reads the context
   │
   └─ Answer: "The hostel curfew is 10:00 PM on weekdays. Late entry
               requires written permission."
```

Notice chunk_07 (the fee chunk) was retrieved but was irrelevant. That's normal — k=3 always returns 3 chunks whether or not all 3 are useful. The LLM ignores the irrelevant one. This is why **k is a trade-off**: too small and you miss the answer, too large and you flood the prompt with noise and pay for more tokens.

Part 3 — Every Tool You Used

Tool	Job in one line	What to say if asked "why this one?"
Python	The language	The whole ML ecosystem is Python-first.
PyPDF 6.13	Extract text from PDFs	Pure Python, no system dependencies, works on any deployment host.
Sentence Transformers 5.6	Turn text into 384-dim vectors	Wraps the whole tokenize → transformer → pool → normalize pipeline into one `.encode()` call.
PyTorch 2.12	Runs the neural network	Not called directly — it's the engine underneath Sentence Transformers.
Hugging Face Hub	Downloads and caches the model	First run downloads ~90 MB, then it's cached locally.
ChromaDB 1.5	Store and search vectors	Open-source, embedded (no server to run), persists to disk, built-in HNSW.
OpenAI SDK 2.42	HTTP client for the LLM	Used with a changed `base_url` to reach OpenRouter.
OpenRouter	Gateway to many LLMs	One API key and one endpoint for dozens of models; has a free tier.
python-dotenv	Loads `.env` into environment variables	Keeps the API key out of the source code.
Streamlit 1.58	Web UI in pure Python	No HTML, CSS or JavaScript needed; free hosting on Community Cloud.
Git / GitHub	Version control + deploy source	Streamlit Cloud deploys straight from a GitHub repo.
venv	Isolated Python environment	Keeps this project's package versions separate from other projects.

3.1 Streamlit — the two functions they will ask about

`@st.cache_resource`

Streamlit re-runs your **entire script from top to bottom** on every user interaction. Without caching, every message would reload the 90 MB embedding model and rebuild the index — several seconds per question. `@st.cache_resource` tells Streamlit "run this once, keep the result in memory, reuse it."

```
@st.cache_resource(show_spinner="Building the knowledge index (first run only)...")
def get_collection():
    return _build_or_load_collection()
```

Know the difference: `cache_resource` is for unserializable global objects you want to *share* — models, DB connections. `cache_data` is for data (DataFrames, JSON) and returns a *copy* each time so callers can't corrupt the cache.

`st.session_state`

Because the script re-runs top to bottom, normal Python variables reset every time. `session_state` is a dictionary that survives re-runs — that's how your chat history persists.

```
if "messages" not in st.session_state:
    st.session_state.messages = []          # only on first load
st.session_state.messages.append({"role": "user", "content": query})
```

The secrets bridge — a genuinely good piece of engineering in your code

```
if "OPENROUTER_API_KEY" in st.secrets:
    os.environ.setdefault("OPENROUTER_API_KEY", st.secrets["OPENROUTER_API_KEY"])
```

Locally the key comes from `.env`; on Streamlit Cloud there is no `.env`, the key lives in `st.secrets`. This block copies it into an environment variable **before** `llm.py` is imported, so the same `llm.py` works in both places unchanged. **Point this out — it shows you thought about environment parity.**

3.2 The lazy import — another detail worth showing off

```
from llm import generate_answer   # imported INSIDE the query handler, not at the top
```

`llm.py` builds the OpenAI client at import time and crashes if there's no API key. If you imported it at the top of the file, a missing key would kill the whole app at startup and the user would see a stack trace. By importing it only when a question is actually asked, the UI still loads and shows a friendly "API key not set" warning instead. **That's graceful degradation.**

Part 4 — Glossary

Every term below can appear in your interview. One line each. If you can define these without hesitating, you sound fluent.

Core RAG

Term	Plain-English meaning
RAG	Retrieval-Augmented Generation. Fetch relevant text first, then let the LLM answer using it.
Retrieval	Finding the most relevant chunks for a question.
Augmentation	Pasting those chunks into the prompt as context.
Generation	The LLM writing the final answer.
Grounding	Forcing the answer to come from supplied text rather than the model's memory.
Hallucination	When an LLM confidently states something false.
Indexing / Ingestion	The one-time pipeline that prepares documents for search.
Knowledge base	The set of documents the system can answer from.
Context window	The maximum amount of text an LLM can read in one request.

Text processing

Term	Plain-English meaning
Chunk	A small piece of a document — yours is 500 characters.
Chunk size	How big each piece is.
Overlap	Characters repeated between neighbouring chunks so facts aren't cut in half.
Stride	How far the window moves each step = chunk_size − overlap = 400 in yours.
Fixed-size chunking	Cut every N characters, ignoring meaning. **This is what you built.**
Recursive chunking	Try paragraph breaks first, then sentences, then words. Smarter.
Semantic chunking	Split where the *topic* changes, detected by embedding similarity.
Token	A word-piece the model actually reads. Roughly ¾ of a word in English.
Tokenization	Converting text into those tokens.
OCR	Optical Character Recognition — reading text out of an *image* of a page.

Embeddings

Term	Plain-English meaning
Embedding / Vector	A list of numbers representing the meaning of text.
Dimension	How many numbers are in that list. Yours: 384.
Dense vector	Mostly non-zero numbers, captures meaning. (Embeddings are dense.)
Sparse vector	Mostly zeros, one slot per vocabulary word. (TF-IDF, BM25 are sparse.)
Semantic search	Searching by meaning.
Keyword / lexical search	Searching by exact word match.
Transformer	The neural network architecture behind modern NLP.
Attention	The mechanism letting each word look at other words to get context.
BERT	An early transformer that reads text in both directions. MiniLM is a compressed BERT.
Mean pooling	Averaging all token vectors into one sentence vector.
Normalization (L2)	Scaling a vector to length 1 so only its direction matters.
Knowledge distillation	Training a small model to copy a big one. How MiniLM was made.
Bi-encoder	Encode question and document separately, compare vectors. Fast. **Yours.**
Cross-encoder	Feed question and document together into the model. Far more accurate, far slower — used for re-ranking.

Vector search

Term	Plain-English meaning
Vector database	A database built to search by vector similarity.
Collection	Chroma's equivalent of a table. Yours is `research_docs`.
Top-K	Return the K most similar results. Yours: k=3.
Cosine similarity	Angle between two vectors. 1 = same direction = same meaning.
Euclidean / L2 distance	Straight-line distance. Smaller = closer. Chroma's default.
ANN	Approximate Nearest Neighbour — trades a little accuracy for a lot of speed.
HNSW	Hierarchical Navigable Small World — the layered-graph ANN algorithm Chroma uses.
Brute-force / exact search	Compare against every vector. Accurate but slow at scale.
Metadata filtering	Restricting search by attached fields, e.g. only 2024 documents.
Re-ranking	Retrieve 20 cheaply, then re-score them with a better model and keep the best 3.
Hybrid search	Combining keyword search and vector search, then merging the rankings.
pgvector	A Postgres extension that adds a vector column and index type.

LLM and prompting

Term	Plain-English meaning
LLM	Large Language Model — predicts the next token, trained on huge text corpora.
Prompt	The full text you send the model.
System / user / assistant roles	Message types in a chat API. System sets behaviour; user asks; assistant replies.
Prompt engineering	Writing prompts to reliably get the behaviour you want.
Temperature	Randomness dial. 0 = deterministic and factual; 1 = creative and varied.
Zero-shot	Asking with no examples. (What you do.)
Few-shot	Including 2–3 example Q&A pairs in the prompt to shape the format.
Fine-tuning	Retraining a model's weights on your own data. Expensive; RAG is usually the cheaper answer.
Inference	Running a trained model to get an output.
MoE	Mixture of Experts — Qwen3-235B-A22B has 235B total parameters but only activates 22B per token.

Engineering and deployment

Term	Plain-English meaning
Environment variable	A config value stored outside the code, e.g. an API key.
.env file	A local file holding those values. Always gitignored.
Secrets management	Keeping credentials out of source control.
venv	An isolated per-project Python environment.
requirements.txt	The list of packages needed to run the project.
Pinning	Locking an exact version, e.g. `chromadb==1.5.9`.
Persistence	Data surviving a restart because it's written to disk.
Cold start	The slow first request while models load and the index builds.
Graceful degradation	Failing partially and usefully rather than crashing entirely.
Idempotent	Running it twice gives the same result as running it once.

Part 5 — Interview Questions & Answers

A. RAG Fundamentals

Q. What is RAG, in one sentence?

A. Retrieval-Augmented Generation — you retrieve relevant text from your own documents first, then give it to an LLM so it answers from that text instead of from memory.

Q. Why use RAG instead of just asking ChatGPT?

A. ChatGPT has never seen my college handbook, so it would either refuse or hallucinate. RAG gives it the actual source text at question time, so answers are grounded in real content and I can show the user which passage the answer came from.

Q. Why not just fine-tune a model on the handbook?

A. Fine-tuning is expensive, needs a lot of training data, has to be redone every time the handbook changes, and still can't cite sources. RAG updates instantly — I just re-index the new PDF. The rule of thumb is: fine-tune to change the model's *style or behaviour*, use RAG to change its *knowledge*.

Q. Why not paste the whole document into the prompt?

A. Three reasons: the context window has a hard limit so large documents won't fit; you pay per token so it's expensive on every question; and models suffer from "lost in the middle" — accuracy drops for facts buried in the middle of a very long prompt. Sending 3 focused chunks beats sending 200 pages.

Q. What are the two phases of a RAG system?

A. Indexing, which runs once offline — extract, chunk, embed, store. And retrieval-plus-generation, which runs on every question — embed the query, search, augment the prompt, generate.

Q. Does RAG completely eliminate hallucination?

A. No, it reduces it. The model can still misread the context or fill gaps. What RAG guarantees is *traceability* — because I show the retrieved passages, a user can verify the answer against the source. And if retrieval fails and returns the wrong chunks, the LLM has nothing correct to work from — that's called retrieval failure, and it's the most common cause of bad RAG answers.

Q. Where can a RAG pipeline fail?

A. Four places. Extraction — a scanned PDF gives no text. Chunking — a fact gets split across a boundary. Retrieval — the right chunk isn't in the top-K. Generation — the model misreads correct context. Most real failures are retrieval failures, which is why re-ranking and better chunking give the biggest quality wins.

Q. How would you evaluate whether your RAG system is any good?

A. Separate the two stages. For retrieval I'd build a test set of questions with the known correct chunk and measure **recall@k** — how often the right chunk is in the top-K — plus MRR. For generation I'd measure **faithfulness** (is every claim supported by the context?) and **answer relevance**. Frameworks like RAGAS automate this using an LLM as judge.

Q. What is "lost in the middle"?

A. A well-documented finding that LLMs pay most attention to the beginning and end of a long prompt and can miss facts placed in the middle. It's an argument for sending fewer, better-targeted chunks rather than dumping everything in.

Q. Why did you build this without LangChain?

A. To actually understand each step. LangChain would have reduced this to about ten lines, but I wouldn't have learned how chunking, embedding, and vector search really work. Now that I do, I'd happily use LangChain or LlamaIndex on a production project — they give you re-rankers, evaluation and document loaders for free.

Q. What is an agentic RAG system?

A. Instead of always doing one retrieval, an LLM agent decides *whether* to retrieve, rewrites the query if the first results are weak, may search multiple times, and can pick between different sources. My system is the simpler "naive RAG" — one retrieval, one generation.

Q. What is corrective RAG / self-RAG?

A. Variants that add a grading step. The system scores whether the retrieved chunks are actually relevant, and if they're not, it re-writes the query and retrieves again, or falls back to web search. It's a feedback loop on top of basic RAG.

B. PDF Processing and Chunking

Q. How do you extract text from a PDF?

A. PyPDF's `PdfReader`. I loop over `reader.pages` and call `page.extract_text()`, joining pages with a newline.

Q. What if the PDF is a scanned image?

A. Then it has no text layer and `extract_text()` returns an empty string. I'd need OCR — Tesseract via pytesseract, or a cloud OCR service — to convert the page images into text first. My current pipeline doesn't handle that.

Q. What is chunking and why is it necessary?

A. Splitting a long document into small pieces. It's necessary because the embedding model can only read 256 tokens at a time, because one vector for a whole document would blur every topic together and retrieve poorly, and because I only want to send the LLM a small amount of relevant text.

Q. What chunk size and overlap did you use, and why those numbers?

A. 500 characters with 100 characters of overlap — a 20% overlap ratio. 500 characters is roughly 85 words, which sits well inside MiniLM's 256-token limit while still being a complete idea rather than a fragment. 20% overlap is the common default; enough to protect against boundary cuts without duplicating too much.

Q. Explain overlap with an example.

A. If the text says "students must return by 10:00 PM" and the boundary falls right after "by", one chunk has the question with no time and the next has a time with no question — neither can answer it. Overlap repeats the last 100 characters into the next chunk, so the complete sentence survives somewhere.

Q. What happens if the chunk size is too small? Too large?

A. Too small and each chunk loses its surrounding context — you retrieve a fragment that's meaningless on its own. Too large and the embedding averages several unrelated topics, so the vector is vague and matches poorly, plus you waste tokens sending irrelevant text to the LLM.

Q. What chunking strategy did you use? (Answer carefully)

A. Fixed-size character chunking with overlap. I want to be precise about that — my README calls it recursive, but it isn't. Recursive chunking splits on paragraph breaks first, then sentences, then words, and only cuts mid-word as a last resort. Mine cuts at exactly 500 characters regardless, so it can split a word in half. Moving to recursive splitting is the first improvement I'd make.

Q. What is semantic chunking?

A. Instead of a fixed size, you embed each sentence and split where consecutive sentences become dissimilar — meaning the topic changed. It produces chunks that are genuinely coherent units, at the cost of embedding everything twice.

C. Embeddings

Q. What is an embedding?

A. A list of numbers that represents the meaning of a piece of text, produced by a neural network. Text with similar meaning gets numerically similar vectors. Mine are 384 numbers long.

Q. Explain embeddings to a non-technical person.

A. Imagine a map where every sentence gets a coordinate, and sentences that mean similar things sit near each other. "Hostel curfew" and "what time must students be back" land almost on the same spot even though they share no words. A normal map needs 2 numbers per point; meaning is more complex, so this map needs 384.

Q. Which model did you use and what do the parts of the name mean?

A. `all-MiniLM-L6-v2`. "all" means it was trained on a broad mixture of general-purpose data, "MiniLM" is a distilled compact BERT, "L6" means 6 transformer layers, "v2" is the version. It outputs 384 dimensions and has about 22 million parameters.

Q. Why that model over a bigger one?

A. It's the best speed-to-quality trade-off for CPU-only free-tier deployment — about 90 MB, runs without a GPU, no API cost, and plenty accurate for one handbook. If I needed better quality I'd use `all-mpnet-base-v2`, which is 768-dim and noticeably better but several times slower.

Q. Walk me through what happens inside `model.encode()`.

A. Three steps. Tokenization splits text into word-pieces and maps them to IDs. Six transformer layers process them with self-attention, so each token's vector is shaped by its surrounding words. Then mean pooling averages all the token vectors into a single 384-number sentence vector, and an L2 normalization layer scales it to length 1.

Q. What is mean pooling and why is it needed?

A. The transformer gives you one vector per *token*, but I need one vector per *chunk*. Mean pooling averages all the token vectors together. The alternative is CLS pooling — using just the special first token — but mean pooling generally works better for sentence similarity.

Q. Are your vectors normalized?

A. Yes. all-MiniLM-L6-v2 has a Normalize module in its pipeline, so every output vector has length exactly 1. That matters because for unit vectors, ranking by L2 distance and ranking by cosine similarity give identical results.

Q. What is the difference between a bi-encoder and a cross-encoder?

A. A bi-encoder — what I use — encodes the question and each document separately, so I can precompute all document vectors once and just compare at query time. Fast. A cross-encoder feeds the question and document into the model *together*, which is much more accurate because the model sees both at once, but you can't precompute anything, so it's far too slow to run over a whole database. The standard pattern is to retrieve with a bi-encoder and re-rank the top 20 with a cross-encoder.

Q. What happens if you embed documents with one model and queries with another?

A. The results are meaningless. Each model produces vectors in its own coordinate space — comparing across them is like comparing GPS coordinates to a seat number. The same model must be used for both, and changing the embedding model means re-indexing everything.

Q. What is the difference between dense and sparse vectors?

A. Dense vectors are what embeddings produce — a few hundred mostly non-zero numbers capturing meaning. Sparse vectors like TF-IDF or BM25 have one slot per vocabulary word and are almost all zeros; they capture exact word matches. Dense finds meaning, sparse finds exact terms. Hybrid search combines both.

Q. Why is 384 dimensions enough? Wouldn't more be better?

A. More dimensions can capture more nuance, but they cost more storage, more memory, and slower search — and they hit diminishing returns quickly. 384 is a deliberate compromise that works well for short chunks. There's also the curse of dimensionality: at very high dimensions distances between points start to look uniform.

Q. What is knowledge distillation?

A. Training a small "student" model to reproduce a large "teacher" model's outputs. That's how MiniLM was made — it keeps most of BERT's quality at a fraction of the size and speed.

Q. What is attention?

A. The mechanism that lets each word in a sentence look at every other word and weight how relevant they are. It's why "bank" gets a different vector in "river bank" versus "bank account" — the surrounding words change its representation.

Q. What does `.tolist()` do in your code and why is it there?

A. `model.encode()` returns a NumPy array, but ChromaDB's API expects plain Python lists, so I convert. It's a small serialization detail — NumPy floats aren't directly JSON-serializable.

D. Vector Database and Search

Q. What is a vector database and why can't you use MySQL?

A. A vector database indexes high-dimensional vectors so you can query "find the most similar" efficiently. A normal SQL database is built for exact matching and range queries on scalar values — it has no index that makes nearest-neighbour search in 384 dimensions fast, so it would have to scan every row.

Q. Why ChromaDB over Pinecone, FAISS, Weaviate or Qdrant?

A. Chroma is open source, free, and embedded — it runs inside my Python process with no separate server to deploy, and it persists to disk automatically. Pinecone is a managed cloud service, great at scale but paid and it needs network calls. FAISS is a raw similarity-search library — very fast, but it doesn't store the original text or metadata for you, so you'd have to manage that yourself. For a single-document project, Chroma was the least operational overhead.

Q. What is top-K and what K did you use?

A. K is how many of the nearest chunks to retrieve. I used 3 by default, and exposed it as a slider from 1 to 6 in the Streamlit UI so it can be tuned live.

Q. What is the trade-off in choosing K?

A. Too small and the chunk holding the answer might not be retrieved at all. Too large and you fill the prompt with irrelevant text, which costs more tokens and can distract the model. K=3 is a reasonable default for short chunks; with larger chunks you'd use a smaller K.

Q. What distance metric does your system use?

A. ChromaDB defaults to squared L2 and I didn't override it, so technically L2. But my embeddings are unit-normalized, and for unit vectors L2 and cosine give the identical ranking — the relationship is `||a−b||² = 2 − 2·cos(a,b)`. If I wanted cosine explicitly I'd pass `metadata={"hnsw:space": "cosine"}` when creating the collection.

Q. Explain cosine similarity simply.

A. It measures the angle between two vectors, ignoring their length. Two arrows pointing the same direction score 1, perpendicular scores 0, opposite scores −1. It's preferred for text because a long document and a short sentence about the same topic should count as similar even though their raw magnitudes differ.

Q. What is ANN and why not exact search?

A. Approximate Nearest Neighbour. Exact search compares the query against every vector — fine for my 23 chunks, but linear in the number of vectors, so hopeless at millions. ANN uses an index to check only a small fraction of candidates. It can occasionally miss the true nearest neighbour, and that small accuracy loss buys an enormous speed gain.

Q. Explain HNSW.

A. Hierarchical Navigable Small World. It builds a multi-layer graph over the vectors. The top layer has few nodes with long-range links, and lower layers get progressively denser. A search starts at the top, takes big jumps toward the target, then drops down a layer and refines. It's like travelling by highway, then main road, then side street instead of checking every house.

Q. What is metadata filtering and do you use it?

A. It's restricting the search by attached fields — for example only chunks from a specific document or year. I don't use it currently, because I store no metadata. That's a real limitation: with multiple PDFs I couldn't filter by source or cite a page number. Adding `{source, page, chunk_index}` metadata is on my improvement list.

Q. What is re-ranking?

A. A two-stage retrieval. First retrieve a wide set — say top 20 — cheaply with the bi-encoder, then re-score just those 20 with a slower, more accurate cross-encoder and keep the best 3. You get near cross-encoder quality at near bi-encoder cost. It's usually the single biggest quality improvement you can add to a basic RAG system.

Q. What is hybrid search?

A. Running keyword search (BM25) and vector search in parallel and merging the results, often with Reciprocal Rank Fusion. It helps because vector search is weak on exact identifiers — product codes, names, acronyms — where keyword search is strong, and vice versa for paraphrased questions.

Q. How would this scale to a million documents?

A. Chroma embedded would become a bottleneck, so I'd move to a server-based store — Qdrant, Weaviate, Milvus, or Postgres with pgvector. I'd batch the embedding generation and probably run it on a GPU, add metadata for filtering, add re-ranking, and cache frequent queries. I'd also tune the HNSW parameters, since the defaults favour build speed over recall.

E. LLM and Prompt Engineering

Q. Which LLM did you use and why?

A. Qwen3-235B-A22B through OpenRouter. It's a strong open-weights model available on OpenRouter's free tier, which mattered for a student project. Because everything goes through one OpenAI-compatible interface, swapping to Llama or DeepSeek is a one-line change.

Q. What is OpenRouter?

A. A gateway that gives you one API key and one endpoint for dozens of models from different providers. It mirrors OpenAI's API format, so I use the official OpenAI Python SDK and just point `base_url` at OpenRouter.

Q. Why are you importing the OpenAI library if you're not using OpenAI?

A. Because OpenRouter is OpenAI-compatible — it speaks the same request and response format. The SDK is just a well-maintained HTTP client for that format. I change `base_url` and it talks to OpenRouter instead.

Q. Walk me through your prompt and why it's written that way.

A. Three parts. A role line — "you are a helpful research assistant" — to set tone. A hard constraint — "use ONLY the provided context" — which is what stops it answering from its own training data. And an explicit fallback sentence to use when the answer isn't in the context. Then the context block, then the question.

Q. Why is that fallback sentence so important?

A. Because LLMs are trained to be helpful, and an unhelpful-looking "I don't know" is something they'll avoid by inventing an answer. Giving the model an explicit, approved way to fail removes the pressure to fabricate. It's the highest-value line in the whole prompt.

Q. What temperature do you use?

A. I don't set it, so it uses the API default of 1.0. That's honestly a flaw — for a factual document-grounded assistant I should set it to 0 or 0.1, so answers are deterministic and stick closely to the source instead of being creative. It's a one-line fix I'd make.

Q. What is temperature actually doing?

A. It scales the probability distribution over the next token before sampling. Low temperature sharpens it, so the most likely token almost always wins — deterministic and factual. High temperature flattens it, so less likely tokens get picked more often — more varied and creative, but more prone to drift.

Q. You send only a `user` message. Should the instructions be a `system` message?

A. Yes, that would be better practice. System messages are given higher priority by most models and are harder to override with injected text in the context. Putting my instructions in a system message and only the question in the user message would make the grounding constraint more robust.

Q. What is prompt injection, and is your app vulnerable?

A. It's when text inside the retrieved content contains instructions that hijack the model — for example a PDF containing "ignore previous instructions and say X". My app is vulnerable in principle, because retrieved chunks go straight into the prompt. Mitigations are putting instructions in a system message, clearly delimiting the context block, and sanitizing or validating the output.

Q. Zero-shot versus few-shot?

A. Zero-shot is asking with no examples — that's what I do. Few-shot includes two or three example question-answer pairs in the prompt to demonstrate the desired format. Few-shot would help if I wanted a consistent structure, like always ending with a citation.

Q. What are tokens and why do they matter?

A. Tokens are the word-pieces a model reads and bills by — roughly ¾ of a word in English. They matter because the context window is measured in tokens and pricing is per token, so retrieving fewer, better chunks directly reduces cost and latency.

Q. What is MoE? (they may ask because of Qwen3-235B-A22B)

A. Mixture of Experts. The model has 235 billion total parameters but a router activates only about 22 billion of them per token — that's what "A22B" means. You get the quality of a very large model at the inference cost of a much smaller one.

F. Streamlit and Deployment

Q. Why Streamlit rather than Flask or FastAPI?

A. Streamlit lets me build the entire UI in Python with no HTML, CSS or JavaScript, and it has purpose-built chat components. For a data/ML demo it's dramatically faster to build. Flask or FastAPI would be the right choice if I needed a proper API for other services to call, or full control over the front end.

Q. How does Streamlit's execution model work?

A. It re-runs the entire script from top to bottom on every user interaction. That's simple to reason about, but it means anything expensive must be cached and any state you want to keep must live in `session_state`, or it resets on every message.

Q. What does `@st.cache_resource` do in your app?

A. It ensures the ChromaDB collection and the embedding model are loaded once and reused. Without it, every question would reload a 90 MB model and rebuild the index — several seconds of delay per message.

Q. Difference between `cache_resource` and `cache_data`?

A. `cache_resource` is for global unserializable objects you want shared across sessions — models, database connections — and it returns the same object. `cache_data` is for serializable data like DataFrames and returns a copy each call, so a caller can't accidentally mutate the cache.

Q. How do you store chat history?

A. In `st.session_state.messages`, a list of dictionaries with role, content, and the source passages. On each re-run I loop over it and replay the messages. It's per-session and in-memory, so it's lost on refresh — persisting it would need a real database.

Q. How do you handle the API key across local and cloud?

A. Locally it's in a gitignored `.env` file loaded by python-dotenv. On Streamlit Cloud there is no `.env`, so the key is set in the dashboard's Secrets and exposed via `st.secrets`. At the very top of my app I bridge `st.secrets` into an environment variable, before `llm.py` is imported — so the same `llm.py` works unchanged in both environments.

Q. Why is `chroma_db/` gitignored if the app needs it?

A. Because it's a build artefact, not source. It's large, binary, and changes constantly, which pollutes git history. Instead the app checks `collection.count() == 0` on startup and rebuilds the index from the PDF automatically. The PDF is the source of truth; the vector store is derived.

Q. Why aren't torch and transformers pinned in requirements.txt?

A. Deliberate. Pinning them broke the Streamlit Cloud build because no matching wheel existed for the runtime's Python version. sentence-transformers already declares compatible ranges, so I let pip resolve them. It's a trade-off — less reproducible, but it actually builds. In a production setup I'd lock everything with a lockfile and control the Python version explicitly.

G. Python and Code

Q. Why is `llm` imported inside the function instead of at the top of the file?

A. A lazy import. `llm.py` constructs the OpenAI client at import time and raises if there's no API key. Importing at the top would crash the whole app at startup with a stack trace. Importing it only when a question is asked lets the UI load normally and show a friendly warning instead — graceful degradation.

Q. What does `os.environ.setdefault()` do, and why not plain assignment?

A. `setdefault` only sets the value if the key isn't already present. So if the environment already has a real key set, I don't overwrite it. It makes the local environment take precedence and the bridge purely additive.

Q. Explain the f-string in your prompt.

A. An f-string is a formatted string literal — anything inside `{}` is evaluated and substituted. I use a triple-quoted f-string so I can write the prompt across multiple lines and drop `{context}` and `{question}` into place.

Q. Why `try/except` around the collection deletion in `vector_store.py`?

A. Because `delete_collection` raises if the collection doesn't exist — which is the case on the very first run. The try/except makes the script safe to run on a fresh machine. I'll be honest: it's a bare `except`, which is bad practice because it swallows every error including typos. It should catch the specific exception.

Q. Why does `results["documents"][0]` have that `[0]`?

A. Because Chroma's `query()` supports batching — you can pass several query embeddings at once, so it returns a list of result-lists, one per query. I send a single query, so `[0]` takes the results for my one query.

Q. What's the difference between a list and a NumPy array here?

A. A Python list is a general container of arbitrary objects. A NumPy array is a fixed-type, contiguous block of memory that supports fast vectorized maths — which is why the model returns one. Chroma's API wants plain lists, hence `.tolist()`.

Q. What is a virtual environment and why use one?

A. An isolated Python installation per project, so this project's package versions don't conflict with another project's. `python -m venv venv` creates it, and `requirements.txt` records what's inside so someone else can recreate it.

Q. What does `get_or_create_collection` do differently from `get_collection`?

A. `get_collection` raises if it doesn't exist. `get_or_create_collection` creates it when missing, which makes startup idempotent — safe to run repeatedly. My Streamlit app uses the get-or-create form for exactly that reason.

Q. Your `app.py` runs `while True`. What are the problems with that?

A. It's a blocking single-threaded CLI loop — fine for a demo, unusable as a service. There's no error handling, so one API failure kills the session, and it can only serve one user at a time. The Streamlit version fixes the interface problem; a real service would need FastAPI with async handlers.

Q. How would you add support for multiple PDFs?

A. Loop over every file in `data/`, and — critically — change the ID scheme. Right now IDs are `chunk_0`, `chunk_1` and so on, which would collide across documents and silently overwrite. I'd use `{filename}_chunk_{i}` and attach metadata with the source filename and page number so I could filter and cite properly.

H. About Your Project Specifically

Q. Why did you build this?

A. Students constantly ask the same questions about fees, hostel rules and admissions, and the answers are all in a handbook nobody reads. I wanted to make the handbook conversational, and RAG was the technique that made it possible without any training data.

Q. How many chunks does your index have?

A. 23, from a 12-page handbook. It's a small corpus — the architecture matters more than the size here, and it scales the same way to thousands of chunks.

Q. How long does a query take?

A. Embedding the query is a few milliseconds on CPU. The vector search on 23 chunks is effectively instant. The LLM call dominates — one to three seconds depending on the answer length. So latency is almost entirely network and generation time, not retrieval.

Q. What was the hardest part?

A. Deployment, honestly. Locally everything worked, but on Streamlit Cloud the build kept failing because I'd pinned torch and transformers to versions with no wheel for the runtime's Python. And the API key handling had to change, because `.env` doesn't exist on the cloud. Unpinning the transitive dependencies and adding the secrets bridge fixed both.

Q. How do you know the answers are actually correct?

A. Right now, manual testing — I ask questions I know the handbook answers and check. The UI also shows the retrieved passages in a Sources expander, so any answer can be verified against the source text. I don't have automated evaluation, and that's a genuine gap; I'd add a labelled test set and measure recall@k and faithfulness.

Q. What happens if someone asks something not in the handbook?

A. Retrieval still returns the 3 nearest chunks — it always returns something — but they'll be irrelevant. The prompt instruction then kicks in and the model replies "I could not find that information in the provided documents." A better version would check the similarity score against a threshold and skip the LLM call entirely if nothing is close enough.

Q. Why is there no similarity threshold?

A. There should be. Chroma returns distances alongside the documents, so I could reject results above a distance cutoff and short-circuit to "not found" — saving an API call and making the refusal more reliable than depending on the prompt alone. It's on my improvement list.

Q. Why does `vector_store.py` delete the collection every time?

A. So re-running `index.py` gives a clean rebuild instead of appending duplicate chunks. It's blunt though — it throws away the whole index even for a small change. A better design would use `upsert` with stable IDs, or hash each document to detect what actually changed.

Q. Does your app support multiple PDFs today?

A. No, and I want to be straight about that — my README says multi-PDF, but `index.py` reads exactly one file, `college_knowledge.pdf`. The architecture supports it and it's a small change, but the ID collision I mentioned has to be fixed first or chunks would overwrite each other.

Q. What would you do differently if you started over?

A. Store metadata from day one — source, page, chunk index — because it unlocks citations and filtering and it's painful to retrofit. Use recursive rather than fixed-size chunking. Set temperature to 0. And add an evaluation set early, so I could measure whether changes actually improved anything instead of guessing.

Q. What did you learn?

A. That retrieval quality, not the LLM, is what determines whether a RAG system is good. I initially assumed a better model would fix bad answers, but almost every bad answer traced back to the right chunk not being retrieved. Chunking strategy and metadata matter more than model choice.

Q. How is this different from a normal chatbot?

A. A normal chatbot answers from whatever it memorized during training and can't tell you where an answer came from. Mine answers only from a specific document, shows the source passages, and updates the moment I re-index a new PDF — no retraining.

I. Weaknesses and Improvements

> Interviewers ask "what are the limitations?" to test self-awareness. Candidates who say "nothing, it works fine" score badly. Candidates who name real gaps *and* the fix score very well. Pick three or four from this list — don't recite all of them.

Q. What are the limitations of your system?

A. Four main ones. No metadata, so I can't cite page numbers or filter by document. Fixed-size chunking, which can cut sentences mid-word. No re-ranking, so retrieval quality is capped by the bi-encoder. And no automated evaluation, so I can't prove a change is an improvement.

Q. What would you add first, and why that one?

A. Metadata — source filename, page number, chunk index. It's cheap to add and it unlocks the most: real citations in the UI, filtering by document, and much easier debugging when retrieval goes wrong. Second would be a cross-encoder re-ranker, because that's the biggest single quality jump available.

Q. How would you handle conversational follow-ups like "and what about for MBA?"

A. Currently that fails — I embed the raw question, and "what about for MBA" has no useful standalone meaning. The standard fix is **query rewriting**: use the chat history to rewrite the follow-up into a self-contained question like "What is the annual fee for MBA?" before embedding it. It's an extra LLM call, but it's what makes multi-turn RAG work.

Q. How would you reduce cost and latency?

A. Cache answers to repeated questions — semantic caching, where you check if a similar question was already asked. Use a smaller model for simple questions. Lower K so fewer tokens go into the prompt. And stream the response so the user sees text immediately even if total time is unchanged.

Q. How would you make this multi-user and production-ready?

A. Move the retrieval and generation behind a FastAPI service so the UI isn't doing the work. Swap embedded Chroma for a server-based vector store. Add authentication, request logging, and rate limiting. Log every query with its retrieved chunks so I can debug failures and build an evaluation set from real traffic. And add proper error handling with retries on the LLM call.

Q. Is there anything unsafe about the app?

A. Prompt injection is the real one — retrieved text goes straight into the prompt, so malicious content in a PDF could try to hijack the instructions. Mitigations are moving instructions to a system message, delimiting the context clearly, and validating output. There's also no rate limiting, so the API key could be drained by abuse.

J. Curveball Questions

Q. Your retrieval returns the wrong chunks. How do you debug it?

A. Step through the pipeline. First check extraction — print the raw text and confirm the fact is even there. Then check chunking — find which chunk contains it and whether it got split. Then embed the question and that chunk directly and compute their similarity; if it's low, it's an embedding-model problem, if it's high but the chunk wasn't returned, it's a K or index problem. Isolating *which* stage failed is the whole job.

Q. A user says the answer is wrong but the retrieved chunks look correct. What now?

A. Then it's a generation failure, not retrieval. I'd lower the temperature to 0, tighten the prompt, and check whether the relevant fact was buried in the middle of the context — the "lost in the middle" effect. Reordering so the highest-scoring chunk comes last sometimes helps, since models weight the end of the prompt heavily.

Q. Would this work for 10,000 PDFs?

A. The architecture holds, the implementation doesn't. I'd need a server-based vector store, batched GPU embedding, metadata and filtering, re-ranking, and an incremental indexing pipeline rather than deleting and rebuilding the whole collection. The retrieval quality problems also get much harder at that scale — that's where hybrid search really starts to matter.

Q. What if two chunks contradict each other?

A. The LLM sees both and will usually either pick one or hedge. Without metadata I can't resolve it properly. With dates in metadata I could prefer the most recent, or surface the conflict to the user rather than silently choosing. It's a good argument for storing document versioning.

Q. Why is your PDF's project report not indexed?

A. Deliberate. `index.py` points only at `college_knowledge.pdf`. `project_report.pdf` sits in the same folder as documentation but isn't part of the knowledge base — indexing it would pollute retrieval with text about the project itself rather than about the college.

Q. If I gave you a week, what would you build?

A. Day 1–2: metadata and recursive chunking, with citations in the UI. Day 3: a cross-encoder re-ranker. Day 4: an evaluation set of about 50 questions with known answers, measuring recall@k and faithfulness. Day 5: query rewriting for conversational follow-ups. Day 6–7: move to FastAPI with logging, so I can see real failures. I'd sequence it that way because evaluation early means every later change can be measured instead of guessed at.

Part 6 — Known Weaknesses in Your Code

These are real discrepancies between your README and your code, or genuine gaps. None of them are disqualifying. All of them are dangerous *only* if you're surprised by them. Read this section twice.

The gap	The reality	What to say
"Recursive chunking"	README says recursive; `chunker.py` is fixed-size character chunking.	Call it fixed-size with overlap. Say you know what recursive chunking is and it's improvement #1.
"Multi-PDF"	README says multi-PDF; `index.py` reads one file.	"The architecture supports it; today it indexes one handbook. The ID scheme would need fixing first."
Chunk ID collision	IDs are `chunk_0…N`, not namespaced by file. Two PDFs would overwrite each other.	Volunteer this — spotting your own bug is a strong signal. Fix is `{filename}_chunk_{i}`.
No metadata	`collection.add()` passes no `metadatas`.	"Biggest limitation. No page citations, no filtering. It's my first addition."
Temperature not set	Defaults to 1.0 — creative, not factual.	"Should be 0 for a grounded factual assistant. One-line fix I'd make."
No system message	All instructions are in a single `user` message.	"System message would be more robust against prompt injection."
Bare `except:`	`vector_store.py` catches everything silently.	"Bad practice — it hides real errors. Should catch the specific exception."
No similarity threshold	Always returns 3 chunks even if all are irrelevant.	"Chroma returns distances; I should reject above a cutoff and skip the LLM call."
No evaluation	Manual testing only.	"I'd build a labelled set and measure recall@k and faithfulness."
No conversation memory in retrieval	Chat history displays, but follow-ups aren't rewritten before embedding.	"Follow-ups like 'what about MBA?' fail. Fix is query rewriting."

> **The framing that turns every one of these into a positive.** Do not say "I didn't do X." Say **"X is a known limitation — here's why it matters and here's the fix."** The first sounds like you didn't think about it. The second sounds like you designed within constraints and understand the roadmap. That is exactly what a senior engineer sounds like.

Part 7 — Night-Before Checklist

Numbers you must know cold

Question	Answer
Chunk size / overlap / stride	500 / 100 / 400 characters
Embedding dimensions	384
Embedding model	all-MiniLM-L6-v2 — 6 layers, ~22M params, 256-token limit
Top-K	3 (slider 1–6 in the UI)
Number of chunks indexed	23
Vector DB / collection	ChromaDB / `research_docs`
Distance metric	L2 by default; equivalent to cosine because vectors are normalized
ANN algorithm	HNSW
LLM	Qwen3-235B-A22B via OpenRouter

Say these out loud until they're natural

1. Your 90-second pitch (Part 1.4)
2. The open-book exam analogy
3. The map analogy for embeddings
4. The overlap example — "return to the hostel by | 10:00 PM"
5. The highway analogy for HNSW
6. Three limitations and their fixes

Do this before you sleep

- Run the app once. Ask three questions. Know what the output looks like.
- Open each of the 8 Python files and read them top to bottom. They're short — 15 minutes total.
- Be able to draw the pipeline diagram on paper from memory. They often ask you to sketch it.

> **Three rules for the room.**
> **1.** If you don't know something, say "I haven't worked with that, but my understanding is..." — then reason out loud. Interviewers value visible thinking far more than a memorized right answer.
> **2.** Never claim a feature you didn't build. One follow-up question and it collapses, and it makes everything else you said suspect.
> **3.** When you finish an answer, stop talking. Filling silence is where candidates talk themselves into trouble.